Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 11 February 2014 - Part I: Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY COMMENTED - THE CANDIDATE SHOULD CORRECTLY ANSWER AND REASONABLY EXPLAIN WHY

WARNING - THE SYNTAX ANALYSIS EXERCISES MUST BE SOLVED BY THE ELR / ELL THEORY

NAME: _____

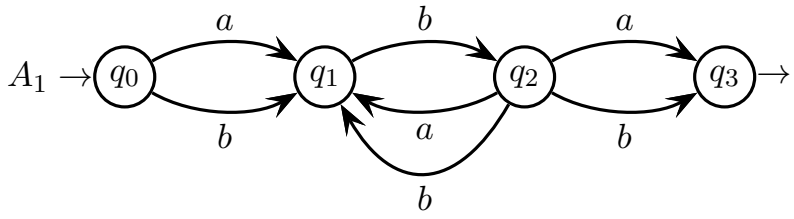
MATRICOLA: SIGNATURE: _____

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
  - I (80%) Theory:
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing
    4. translation and semantic analysis
  - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the finite automaton A_1 shown below:



and the following regular expression e_2:

$$e_2 = (\, b\, b\,)^*\ a$$

Answer the following questions, and explain the obtained results:

(a) By means of the *BMC* method (node elimination), write a regular expression e_1 such that the language equality $L\,(e_1) = L\,(A_1)$ holds. If possible, modify the regular expression e_1 to make it more compact.

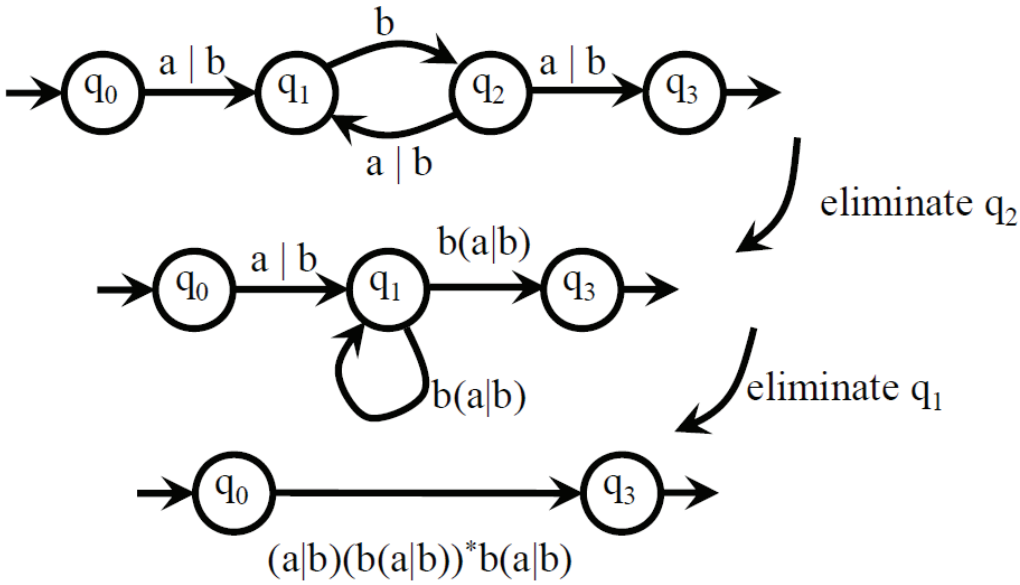
(b) By means of the *BS* method (Berry-Sethi), build a deterministic finite automaton A_2 that recognizes language $L\,(e_2)$.

(c) By means of the automaton product construction (cartesian product), build a finite automaton A_3 that recognizes the intersection language $L\,(A_1) \cap L\,(A_2)$.

(d) If necessary, minimize the state number of the automaton A_3 found before, by means of a systematic method.

Solution

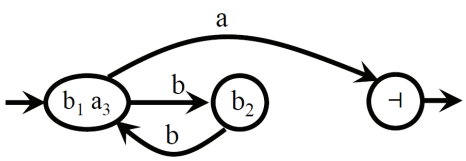
(a) Here is the regular expression e_1:



$$(a|b)(b(a|b))^*b(a|b)$$
$$=$$
$$(\,(a \mid b)\ b)^+\,(a \mid b)$$

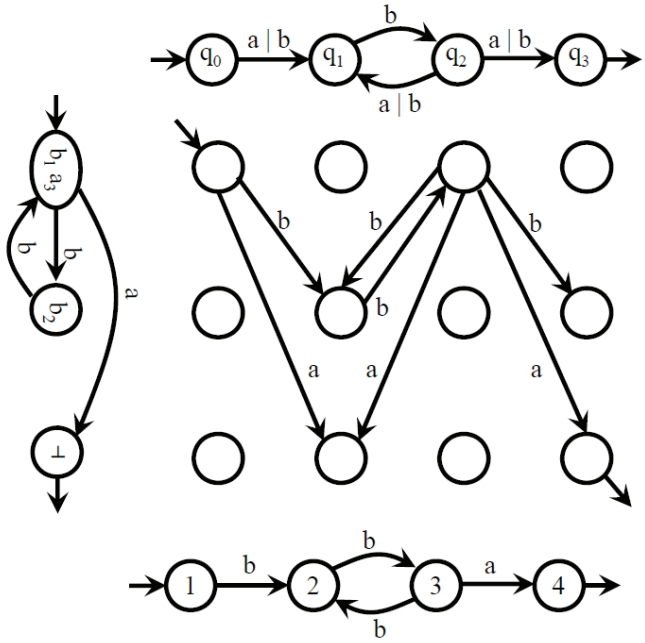
(b) Here is the deterministic automaton A_2 that recognizes language $L\,(e_2)$.

$\mathrm{Ini}(e_2 \dashv) = \mathrm{Ini}((b_1 b_2)^* a_3 \dashv) = \{b_1\ a_3\}$

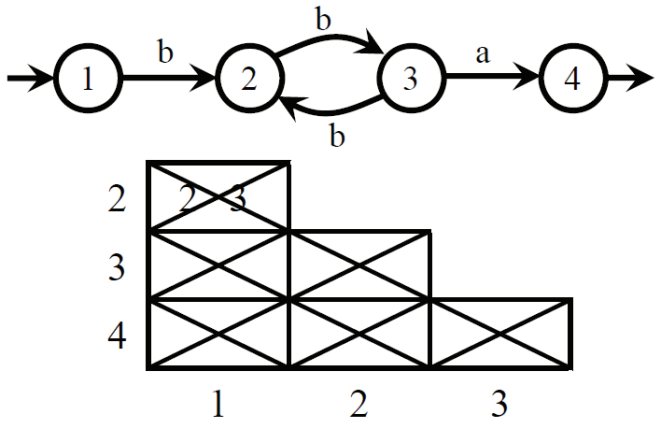


x	Fol(x)
b_1	b_2
b_2	$a_3\ b_1$
a_3	$\dashv$

(c) Here is the automaton A_3 that recognizes the intersection language $L(A_1) \bigcap L(A_2)$.



(d) The automaton A_3 has already a minimum number of states:



2 Free Grammars and Pushdown Automata 20%

1. Consider the following language L, over the alphabet $\{ a, b, c \}$:

$$L = \{ a^p \, b^q \, c^{p+q} \, c^* \mid \quad p, q \geq 0 \}$$

Answer the following questions:

(a) Write a grammar G that generates language L. The grammar must be not ambiguous and must be in the not extended form (BNF).

(b) (optional) Consider a new language L', over the same alphabet as language L, defined in this way:

$$L' = (\neg L) \cap (a^+ \, b^+ \, c^+)$$

that is, defined as the intersection between the complement of L and a regular expression. For such a new language L', write a free grammar G' of any type, but not in the extended form (BNF).

Solution

(a) Slightly change the representation of a generic string of language L into the equivalent one $a^p\ b^q\ c^*\ c^{p+q}$. Then it is easy to think of a decomposition of L into known components, such as this one:

$$L = a^p \left(b^q\ (\,c^*\,)\ c^q \right) c^p \qquad p,\, q \geq 0$$

where the number of letters c is $p + q + * \geq 0$, and clearly it is greater than or equal to $p + q$. Here the components are nested structures such as $a^n\ c^n$ and $b^n\, c^n$ (with $n \geq 0$), and regular ones such as c^*, which can all be generated unambiguously. Therefore a *BNF* grammar G that fulfills the specifications and is unambiguous, is as follows (axiom S):

$$G \begin{cases} S \;\rightarrow\; a\ S\ c \mid B & \text{- - yields } a^p\ B\ c^p \text{ for } p \geq 0 \\ B \;\rightarrow\; b\ B\ c \mid C & \text{- - yields } b^q\ C\ c^q \text{ for } q \geq 0 \\ C \;\rightarrow\; C\ c \mid \varepsilon & \text{- - yields } c^* \end{cases}$$

The axiom S generates the outer nested structure, then the nonterminal B generates the inner one, and finally the nonterminal C generates the regular one, as known. Grammar G is unambiguous as all its substructures are so.

(b) One can preliminarily find a description of language L' in logical terms, i.e., by means of a characteristic predicate. Notice soon that intersecting with the given regular expression, forces the letters to keep the left-to-right ordering a-b-c, and that at least one sample per each letter type has to be present. So the strings of L' look like $a^p\, b^q\, c^r$. Yet due to the complement operation, the number of letters c must be smaller than the sum of the numbers of letters a and b, i.e., the inequality predicate $r < p + q$ must hold; and additionally it must hold $1 \leq p,\, q,\, r$. So a possible logical description of L' is as follows:

$$L' = \{\ a^p\ b^q\ c^r \mid \quad 1 \leq r < p + q\ \wedge\ p,\, q \geq 1\ \}$$

Now writing a grammar G', can be easily done by modifying grammar G, as the language L' has a decomposition similar to that of language L. The main difference is how to enforce the inequality $r < p + q$. A solution is to make the letter c optional in G when generating the two nested structures, as follows:

$$G' \begin{cases} S \;\rightarrow\; a\ S\ c \mid a\ S \mid a\ B & \text{- - yields } a^p\ B\ c^{p'} \text{ for } p > p' \geq 0 \\ B \;\rightarrow\; b\ B\ c \mid b\ B \mid b\ C & \text{- - yields } b^q\ C\ c^{q'} \text{ for } q > q' \geq 0 \\ C \;\rightarrow\; c & \text{- - yields } c \end{cases}$$

and to have at least one letter a and one letter b in the nested structures, as well as to have exactly one letter c in the regular structure, in the obvious way. In general grammar G' generates a string model $a^p\, b^q\, c\, c^{q'}\, c^{p'} = a^p\, b^q\, c^{p'+q'+1} = a^p\, b^q\, c^r$; so with the above inequalities it holds $r = p' + q' + 1 \leq (p-1) + (q-1) + 1 = p + q - 1 < p + q$, as requested. This version of G' is straightforward, but of course it is ambiguous, as for instance the valid string $a\,a\,b\,b\,c\,c$ of L' has two syntax trees: $\underbrace{a\,a\,b\,b\,c\,c}_{S}$ and $a\,a\,\underbrace{b\,b\,c\,c}_{B}$.

An unambiguous version can be found by refining grammar G'. Notice that in G' all the letters c, but the innermost one, are generated in a free association with letters a and b; it is such a freedom that causes ambiguity. A way to remove it, is to make the grammar generate all the letters c, but the innermost one, at the beginning of the derivation, so that after ceasing, their generation may not be resumed. A grammar G'' that does so, obtained from G', is as follows:

$$G'' \begin{cases} S & \rightarrow & a\,S\,c \mid a\,B \mid a\,S' \mid a\,B' \\ B & \rightarrow & b\,B\,c \mid b\,C \mid b\,B' \\ S' & \rightarrow & a\,S' \mid a\,B' \\ B' & \rightarrow & b\,B' \mid b\,C \\ C & \rightarrow & c \end{cases}$$

The axiom is S. After drifting to the primed nonterminals, a derivation of grammar G'' is unable to generate letters c any more, but the innermost one. So grammar G'' is unambiguous: for instance the latter tree above is impossible.

With some straightforward adjustment, grammar G'' can be simplified into grammar G''', as follows (axiom S):

$$G''' \begin{cases} S & \rightarrow & a\,S\,c \mid a\,B \mid S' \\ B & \rightarrow & b\,B\,c \mid B' \\ S' & \rightarrow & a\,S' \mid a\,B' \\ B' & \rightarrow & b\,B' \mid b\,C \\ C & \rightarrow & c \end{cases}$$

which is unambiguous as well.

2. One wishes one formalized a fragment of a hypothetical programming language, by considering only the logical expressions with the characteristics listed below:

   - The logical expressions contain only the operators "**impl**", i.e., logical implication, and "**not**", i.e., logical negation.
   - Implication is right-associative, thus a logical expression like "α **impl** β **impl** γ" must be interpreted as "α **impl** (β **impl** γ)".
   - Negation has a priority higher than implication, so that a logical expression like "**not** α **impl** β" must be interpreted as "(**not** α) **impl** β".
   - The logical subexpressions may be contained in parentheses (round brackets). This is how to change the predefined precedence order of the logical operators or to change their associativity, like in "(α **impl** β) **impl** γ".
   - The elementary logical expressions, e.g., α, β, ..., are relational expressions of the form "$f = g$", where the members f and g are numerical expressions.
   - The numerical expressions, e.g., f or g, consist either of one term t or of the addition of an arbitrary number k of terms, i.e., "$t_1 + t_2 + \ldots + t_k$".
   - Addition does not have any imposed association order. Therefore the grammar must be defined so that in the syntax tree all the terms of an addition are brother nodes, i.e., all are child nodes of one parent node.
   - A term, e.g., t or t_i, is either the terminal **n**, which represents a generic number, or a subtraction subexpression of the form "$\mathbf{n} - \mathbf{n} - \ldots - \mathbf{n}$", with one minuend and one or more subtrahends.
   - Subtraction has a priority higher than addition, so that a numerical expression like "$\mathbf{n} + \mathbf{n} - \mathbf{n} + \mathbf{n}$" must be interpreted as "$\mathbf{n} + (\mathbf{n} - \mathbf{n}) + \mathbf{n}$", i.e., as the addition of three terms, the middle one being a subtraction subexpression.
   - Subtraction is left-associative, thus a subtraction subexpression like "$\mathbf{n} - \mathbf{n} - \mathbf{n}$" must be interpreted as "$(\mathbf{n} - \mathbf{n}) - \mathbf{n}$".

   Notice that the numerical expressions *may not* contain parentheses.

   Here are a few sample phrases of this language:

   $\mathbf{n} = \mathbf{n} - \mathbf{n} + \mathbf{n} - \mathbf{n} - \mathbf{n} + \mathbf{n}$

   not $\mathbf{n} = \mathbf{n} - \mathbf{n}$

   not not $\mathbf{n} =$ **n impl not** $\mathbf{n} = \mathbf{n}$

   not (**not** $\mathbf{n} =$ **n impl** $\mathbf{n} = \mathbf{n}$)

   Write a grammar that models the syntax described above. The grammar must be not ambiguous and should be in the extended form (*EBNF*).

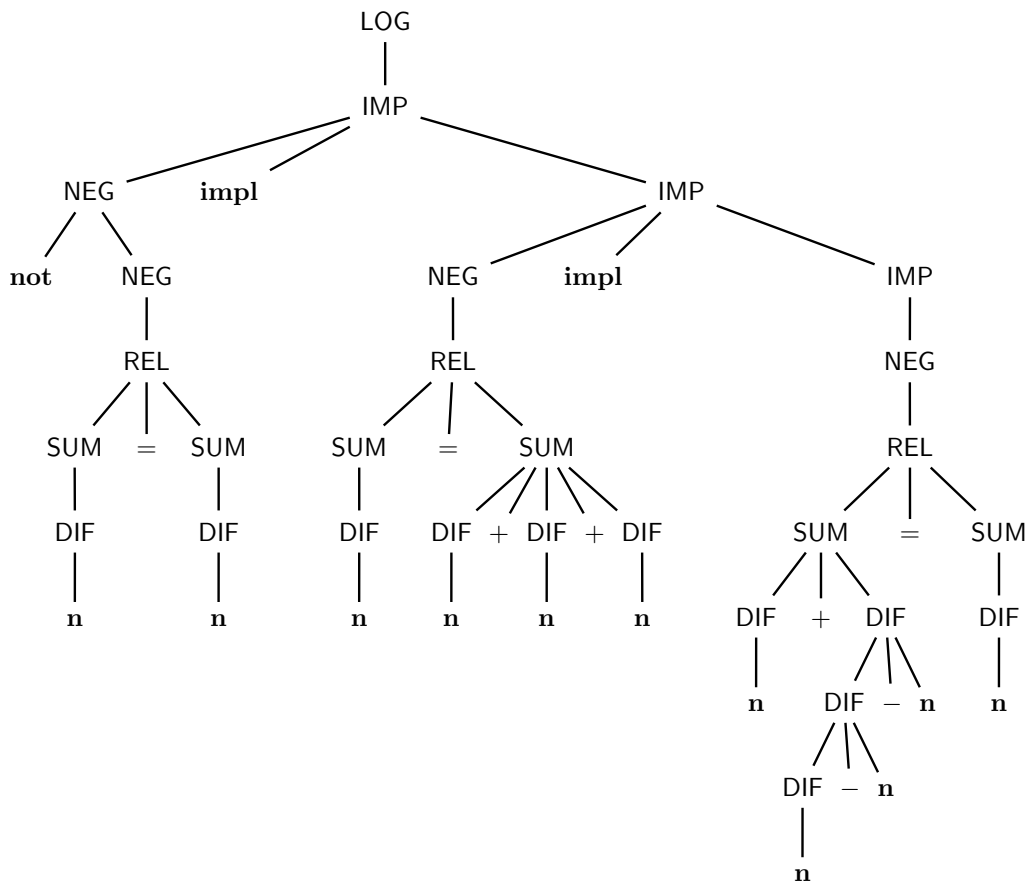
Solution

Here is the grammar requested (axiom LOG_EXP), in the extended form (*EBNF*):

$$
\begin{cases}
1: & \langle \mathsf{LOG_EXP} \rangle & \rightarrow & \langle \mathsf{IMP_EXP} \rangle \\
2: & \langle \mathsf{IMP_EXP} \rangle & \rightarrow & \langle \mathsf{NEG_EXP} \rangle \ \textbf{impl} \ \langle \mathsf{IMP_EXP} \rangle \mid \langle \mathsf{NEG_EXP} \rangle \\
3: & \langle \mathsf{NEG_EXP} \rangle & \rightarrow & \textbf{not} \ \langle \mathsf{NEG_EXP} \rangle \mid \text{`(' } \langle \mathsf{LOG\_EXP} \rangle \text{ `)'} \mid \langle \mathsf{REL_EXP} \rangle \\
4: & \langle \mathsf{REL_EXP} \rangle & \rightarrow & \langle \mathsf{SUM_EXP} \rangle \text{ `=' } \langle \mathsf{SUM_EXP} \rangle \\
5: & \langle \mathsf{SUM_EXP} \rangle & \rightarrow & \langle \mathsf{DIF_EXP} \rangle \ \big(\text{`+'} \langle \mathsf{DIF_EXP} \rangle \big)^* \\
6: & \langle \mathsf{DIF_EXP} \rangle & \rightarrow & \langle \mathsf{DIF_EXP} \rangle \text{ `−' } \mathbf{n} \mid \mathbf{n}
\end{cases}
$$

The proposed grammar is unambiguous as by construction it consists only of unambiguous substructures. The implication rule (2) is right-recursive, for right associativity. The addition rule (5) is iterative (*EBNF*), not recursive, for not having an imposed associativity. The subtraction (or difference) rule (6) is left-recursive, for left associativity. Since rule 3 is invoked by rule 2, negation takes precedence over implication; and since rule 6 is invoked by rule 5, subtraction takes precedence over addition. The categorization axiomatic rule (1) is used only for excluding the axiom from the right parts of the rules, and of course it could be eliminated.

Here is a sample syntax tree (not requested), with nonterminal names shortened:



This sample tree does not include the case of bracketed logical subexpressions, which is quite obvious. The phrase described by the sample tree is the following one:

$$\textbf{not}\ \underbrace{\underbrace{\underset{t}{n}}_{f} = \underbrace{\underset{t}{n}}_{g}}_{\alpha}\ \textbf{impl}\ \underbrace{\underbrace{\underset{t}{n}}_{f} = \underbrace{\underset{t_1}{n} + \underset{t_2}{n} + \underset{t_3}{n}}_{g}}_{\beta}\ \textbf{impl}\ \underbrace{\underbrace{\underset{t_1}{n} + \underbrace{n - n - n}_{t_2}}_{f} = \underbrace{\underset{t}{n}}_{g}}_{\gamma}$$

One clearly sees in the syntax tree that implication is right-associative (subtree growing right), addition is unspecified (flat subtree), subtraction is left-associative (subtree growing left), and that negation and subtraction (lower level) take precedence over implication and addition (upper level), respectively.

So the phrase interpretation given by the sample tree above, is as follows:

$$\big(\,\textbf{not}\ \alpha\,\big)\ \textbf{impl}\ \big(\,\beta\ \textbf{impl}\ \gamma\,\big)$$

and fully expanded, as follows:

$$\big(\,\textbf{not}\ \ (\,n = n\,)\,\big)\ \textbf{impl}\ \Big(\,\big(\,n = (\,n + n + n\,)\,\big)\ \textbf{impl}\ \big(\,(\,n + (\,(\,n - n\,) - n\,)\,) = n\,\big)\Big)$$

and is conformant to the specifications listed in the text.

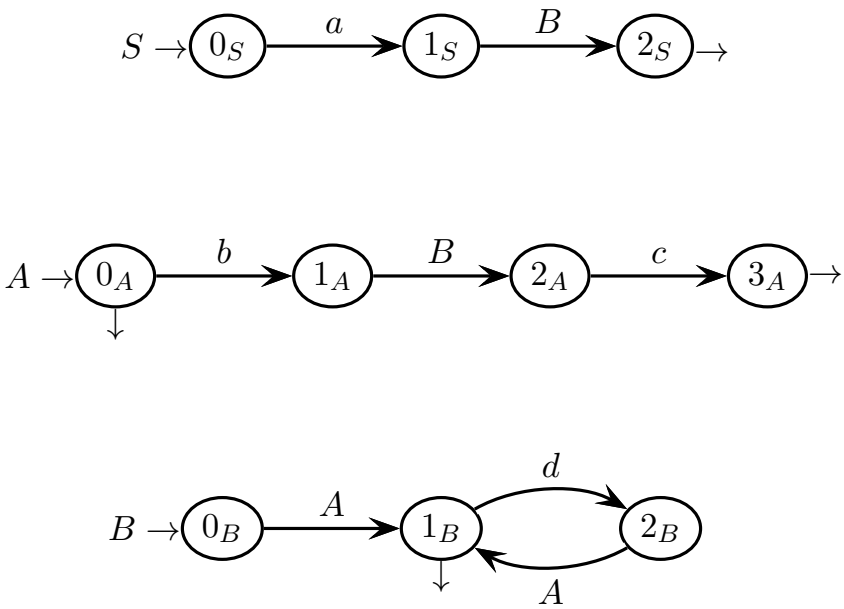
To conclude, notice that rules 2 and 6 could be shortened this way:

$$\begin{cases} 2'\colon\ \ \langle\mathsf{IMP_EXP}\rangle \ \rightarrow\ \langle\mathsf{NEG_EXP}\rangle\ \big[\,\textbf{impl}\ \langle\mathsf{IMP_EXP}\rangle\,\big] \\ 6'\colon\ \ \langle\mathsf{DIF_EXP}\rangle \ \rightarrow\ \big[\,\langle\mathsf{DIF_EXP}\rangle\ \text{`}-\text{'}\,\big]\ \mathbf{n} \end{cases}$$

by resorting to the optionality operator (here rendered with square brackets).

3 Syntax Analysis and Parsing 20%

1. Consider the following grammar G, represented as a network of recursive machines (with axiom S):



Answer the following questions:

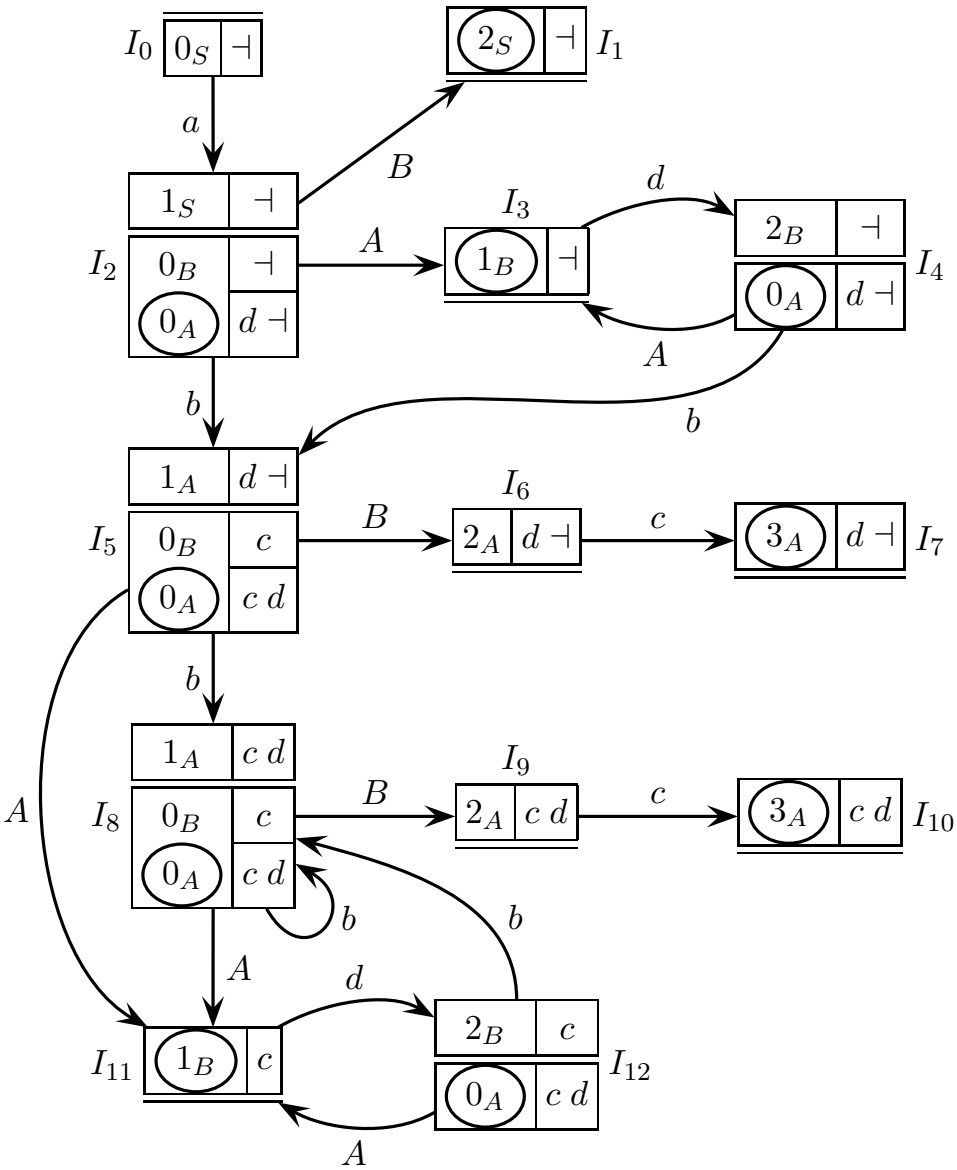
(a) Draw the complete pilot graph of grammar G, say if the grammar has the $ELR(1)$ property, and specify which m-states of the pilot have to be examined to prove the property.

(b) Say if grammar G has the $ELL(1)$ property, and decorate the $PCFG$ graph of the grammar in the points where it is necessary to prove the property.

(c) (optional) Draw the $PCFG$ graph of grammar G, and put all the call arcs and all the guide sets on it (the guide sets must decorate the call arcs and the exit darts of the final nodes).

Solution

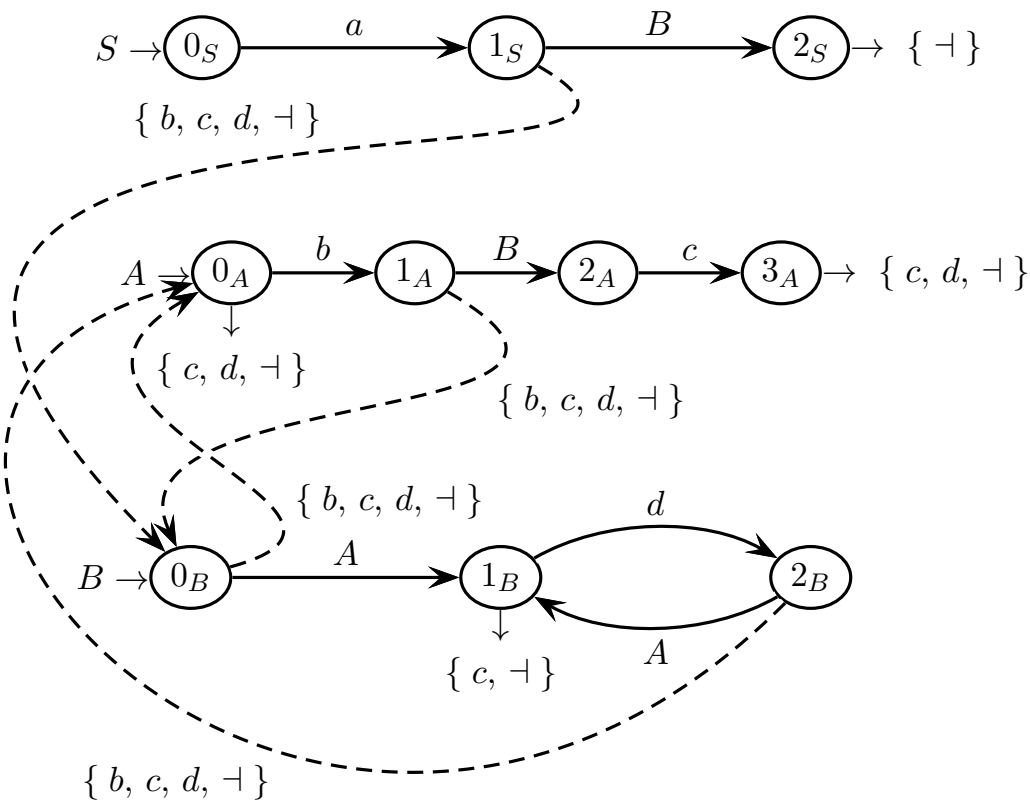
(a) The *ELR* pilot has the *ELR* property (there are no conflicts) and has 13 m-states in total. So the grammar is *ELR*. Only shift-reduce conflicts would be possible in principle, but they are excluded by the look-ahead sets.



By the way, notice that the *ELR* pilot satisfies the *STP* and that the grammar is not left recursive. Therefore, since additionally the pilot satisfies the *ELR* condition, one concludes immediately that the grammar is *ELL* as well.

(b) All the guide sets are reported on the figure included in the answer to point (c). To establish that G is $ELL(1)$, it suffices to remark that the guide set on the exit dart of the final state 0_A is $\{ c, d, \dashv \}$, disjoint from the set $\{ b \}$ on the shift arc $0_A \to 1_A$, and that the guide set on the exit dart of the final state 1_B is $\{ c, \dashv \}$, disjoint from the set $\{ d \}$ on the shift arc $1_B \to 2_B$.

(c) Nonterminals A and B are nullable, while S is not. The *PCFG* has the *ELL* property. So the grammar is *ELL* (as already found before).



Notice that the guide (or prospect) sets on the exit darts that tag the final states in the *ELL* pilot, can be straightforwardly obtained by joining the corresponding look-ahead sets of the reduction states in the *ELR* pilot.

Instead the guide sets on the call arcs must be computed by using the related equations (in the textbook or in the lecture slides). For instance the guide set $\{ b, c, d, \dashv \}$ on the call arc $0_B \dashrightarrow 0_A$ to machine M_A, is computed this way:

- take the letter b on the shift arc from 0_A
- since nonterminal A is nullable, take the letter d on the shift arc from 1_B
- since nonterminal A is nullable and state 1_B is final, take the guide set $\{ c, \dashv \}$ on the exit dart of 1_B

Have a look at the above mentioned equations. Caution: it would be a conceptual error to take the guide set $\{ c, d, \dashv \}$ on the exit dart from 0_A (see the comments in the textbook or in the lecture slides) ! Though, in this particular case the result would be correct (but this just happens by chance).

Consequently the guide set $\{ b, c, d, \dashv \}$ on the call arc $1_S \dashrightarrow 0_B$ to machine M_B, includes the guide set on the call arc $0_B \dashrightarrow 0_A$ to machine M_A, already computed before, because the latter call arc is concatenated to the former one. The same applies to the guide set $\{ b, c, d, \dashv \}$ on the call arc $1_A \dashrightarrow 0_B$ to machine M_B. Instead the guide set on the call arc $2_B \dashrightarrow 0_A$ to machine M_A has to be recomputed, but the result is the same as before: $\{ b, c, d, \dashv \}$; again, remember not to include into it the guide set on the exit dart from 0_A ! Since all these guide sets already contain the entire terminal alphabet plus the terminator $\dashv$, it is no use to go further for more contributions.

4 Translation and Semantic Analysis 20%

1. Consider the following grammar G_s (with axiom A), which generates the Dyck language with one parenthesis type, namely the round brackets:

$$G_s \begin{cases} A \rightarrow \text{`(' } A \text{ `)' } B \mid \varepsilon \\ B \rightarrow \text{`(' } A \text{ `)' } A \mid \varepsilon \end{cases}$$

Answer the following questions:

(a) Write a syntactic transduction scheme G_τ (or a translation grammar), with grammar G_s as its source component. The scheme describes a translation τ such that, considering the series of the pairs of matching parentheses at the same depth level, the pairs placed at an odd position (first, third, etc.) are left unchanged, while those placed at an even position (second, fourth, etc.) are translated into square brackets. For instance:

$$\begin{aligned} \tau\big(\text{“}()()()\text{”}\big) &= ()[]() \\ \tau\Big(\text{“}()\big(()()\big)()\text{”}\Big) &= ()\big[()[]\big]() \end{aligned}$$

(b) (optional) Write a new syntactic transduction scheme $G_{\tau'}$ (or a translation grammar) that translates the round brackets into square ones in the same way as the previous scheme G_τ, but with the additional feature of putting a character “$\bullet$” (a point) into all the bracket pairs (round or square) that do not contain any more parentheses, i.e., into all the empty pairs. For instance:

$$\tau'\Big(\text{“}()\big(()()\big)()\text{”}\Big) = (\bullet)\big[(\bullet)[\bullet]\big](\bullet)$$

A possible approach to write the new scheme $G_{\tau'}$, is the following (but he who prefers, is free to do differently, if he thinks it is better). Take a new source grammar G'_s, a variant of the old one G_s, that uses the following nonterminals:

- S, is the axiom
- A, which generates a parenthesis pair at the first position in the series of pairs at the same depth level
- B, which generates a parenthesis pair at an even position in the series of pairs at the same depth level
- C, which generates a parenthesis pair at an odd position (greater than one) in the series of pairs at the same depth level

Solution

(a) Here is the target grammar G_d (axiom A) that matches the source grammar G_s:

$$G_d \begin{cases} A & \rightarrow & \text{`(' } A \text{ `)' } B \mid \varepsilon \\ B & \rightarrow & \text{`[' } A \text{ `]' } A \mid \varepsilon \end{cases}$$

Clearly nonterminal A generates the brackets structures concatenated at an odd position (first, third, etc.) and nonterminal B those at an even one (second, fourth, etc.). So it suffices to let nonterminal A behave as in the source grammar and to make nonterminal B generate square brackets instead of round ones.

(b) Here is the source grammar G' (axiom S):

$$G'_s \begin{cases} S & \rightarrow & A \mid \varepsilon \\ A & \rightarrow & \text{`(' } A \text{ `)' } B \mid \varepsilon \\ B & \rightarrow & \text{`(' } A \text{ `)' } C \mid \varepsilon \\ C & \rightarrow & \text{`(' } A \text{ `)' } B \mid \varepsilon \end{cases}$$

and here is the matching target grammar G'_d:

$$G'_d \begin{cases} S & \rightarrow & A \mid \varepsilon \\ A & \rightarrow & \text{`(' } A \text{ `)' } B \mid \text{`}\bullet\text{'} \\ B & \rightarrow & \text{`[' } A \text{ `]' } C \mid \varepsilon \\ C & \rightarrow & \text{`(' } A \text{ `)' } B \mid \varepsilon \end{cases}$$

In the source grammar G_s, it is clear that nonterminal A generates only the parenthesis structure in the first (of course odd) position, and that nonterminals B and C alternate in generating the structures subsequently concatenated in the even (second, fourth, etc) and odd (third, fifth, etc) positions, respectively. In the target grammar G_d, it is clear that the brackets are translated as before. Since only nonterminal A occurs inside a bracket pair, and since whenever it ends its derivation it generates a "$\bullet$", then every empty bracket pair will contain a "$\bullet$", and only the empty ones will contain it. The axiomatic rule is only for generating the empty string, which is still permitted.

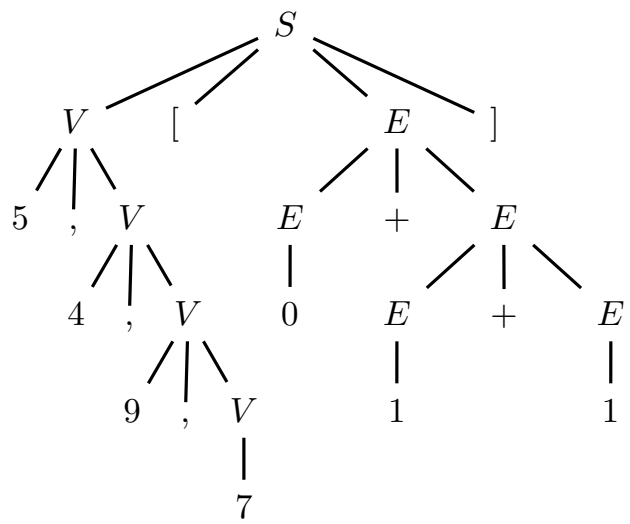
2. Take a vector V of numbers and a summation E of positive or null integers. Consider the problem of finding the following element of V:

```
V [i1 + i2 + ... + ik]
```

where $i1$, $i2$, ..., ik (with $k \geq 1$) are the integers of summation E. Suppose vector V is indexed as in the language C, starting from index 0.

The problem above is modeled by the following syntax (with axiom S), which generates the vector V (modeled as a list of numerical elements) and concatenates to V the summation E (modeled as an expression with addition):

$$\begin{cases} S & \rightarrow & V\ [\ E\ ] \\ V & \rightarrow & n\ ,\ V \\ V & \rightarrow & n \\ E & \rightarrow & E\ +\ E \\ E & \rightarrow & i \end{cases}$$



The terminals n and i schematize a generic element of vector V and a generic integer of summation E, respectively. Commas and brackets are just "syntactic sugar".

For instance suppose vector V has the four elements 5, 4, 9, 7. If one wishes one knew the value of $V[\,0+1+1\,]$, then the result is element 9 of V. The string that models this instance of the problem, is "$\underbrace{5,\ 4,\ 9,\ 7}_{vector}\,[\,\underbrace{0\ +\ 1\ +\ 1}_{summation}\,]$"; see its tree above.

Answer the following questions:

(a) Write an attribute grammar that, as it is requested by the problem above, brings the wanted vector element to the tree root S. Define the necessary attributes and write the semantic functions coupled with the grammar rules (please use the tables already prepared on the next pages). In the semantic functions, use the terminal names n and i to mean the values of the terminals themselves.

(b) Say if the attribute grammar designed is of type one-sweep, and explain why. In the case the grammar is one-sweep, write the semantic evaluation procedure of axiom S, say if the grammar is of type L as well, and explain why.

(c) (optional) If the attribute grammar designed is of type one-sweep, write the semantic evaluation procedure of nonterminal V; otherwise compute a correct evaluation order of the attributes for the sample syntax tree shown above.

attributes to be used for the grammar

type	*name*	*domain*	*nonterm.*	*meaning*

#	*syntax*	*semantics* - to be completed
1:	$S_0 \rightarrow V_1 \; [\; E_2 \;]$	
2:	$V_0 \rightarrow n \; , \; V_1$	
3:	$V_0 \rightarrow n$	
4:	$E_0 \rightarrow E_1 \; + \; E_2$	
5:	$E_0 \rightarrow i$	

Solution

(a) A solution is to use three attributes: a left attribute α for computing E, a right attribute β for propagating the value of E down through vector V and for accessing the wanted vector element, and one more left attribute γ for transporting the element value up to the root. It is easy to see that such an attribute choice and evaluation scheme will allow us to write a one-sweep attribute grammar.

type	*name*	*domain*	*nonterm.*	*meaning*
left	α	integer	E	summation value to be computed and transported upward to the tree root
right	β	integer	V	index value to be propagated downward through the vector
left	γ	integer	V, S	value of the wanted vector element, to be transported upward to the tree root

Here are the semantic functions:

#	*syntax*	*semantics*
1:	$S_0 \rightarrow V_1 \, [\, E_2 \,]$	$\beta_1 = \alpha_2$ $\gamma_0 = \gamma_1$

2:	$V_0 \rightarrow n \, , \, V_1$	$\beta_1 = \beta_0 - 1$
		if $\beta_0 > 0$ **then**
		$\gamma_0 = \gamma_1$
		else if $\beta_0 == 0$ **then**
		$\gamma_0 = n$
		else - - i.e. $\beta_0 < 0$
		$\gamma_0 = nil$
		endif

3:	$V_0 \rightarrow n$	**if** $\beta_0 > 0$ **then**
		vector access error
		else if $\beta_0 == 0$ **then**
		$\gamma_0 = n$
		else - - i.e. $\beta_0 < 0$
		$\gamma_0 = nil$
		endif

4:	$E_0 \rightarrow E_1 + E_2$	$\alpha_0 = \alpha_1 + \alpha_2$
5:	$E_0 \rightarrow i$	$\alpha_0 = i$

Remember that formally all the attributes must be assigned wherever their associated nonterminals occur. Thus attribute γ, which is associated to nonterminal

V, must be assigned in all the inner nodes V of the tree, that is, also after the attribute evaluation flow has walked across the wanted vector element (it happens if attribute β becomes negative); in such cases attribute γ is assigned value *nil*; but in a practical implementation, such a useless assignment ought to be simply omitted. The vector access error can be dealt with, either by stopping the evaluation, or by sending out a diagnostic and still assigning *nil* to γ.

There may be other solutions, more or less similar. For instance one might send down along the vector subtree, the expression value and an index starting from 0 and incrementing at each node. When the index equals the expression value, the terminal is sent up to the root. It needs one more attribute, anyway.

(b) The attribute grammar is one-sweep. This can be argued informally (by words), as done before, or by checking the one-sweep condition. In the latter case: the left attribute α of a parent node E depends on the left attributes α (itself) of its two child nodes E, so it OK; the right attribute β of a child node V depends on the right attribute β (itself) of its parent node V, or (only in the tree root) on the left attribute α of its brother node E, so it is OK; and the left attribute γ of a parent node V (or S) depends on the right attribute β of the parent node V itself (not for S) and on the left attribute γ of its one child node V, so it is OK.

Since the attribute grammar is one-sweep, one can draw the sibling graph of the two child nodes of the axiom S, which is: $V \longleftarrow E$ Now the semantic procedure of S and can be easily designed, provided that the semantic evaluation order is first E and then V (opposite w.r.t. the syntax). Here it is:

procedure S (**out** γ)
  variable
    α', β', γ': **integer**
  call E (α')
  $\beta' := \alpha'$
  call V (β', γ')
  $\gamma := \gamma'$
end procedure

Yet the grammar is not of type L, for there are two obstacles: first, because the grammar is ambiguous (and so it is not *LL* either) due to the two-sided recursion in the rule "$E \rightarrow E \ + \ E$"; and second, because the syntactic ordering of the nonterminals V and E in the axiomatic rule is different (in particular it is reversed) from the semantic evaluation order of these two nonterminals.

The former obstacle could be circumvented by making the grammar unambiguous, for instance by generating the summation with a right recursive rule like "$E \rightarrow n \ + \ E$" (the other rules are left unchanged). Then one would easily see that the grammar becomes *LL*. But the latter obstacle is hard to pass over, unless the language itself is changed by swapping the list of vector elements and the summation; of course, one might instead try to drastically change the attributes and the semantic scheme, though one should pay attention to avoid big attributes and an exceedingly complex scheme; it also seems hard to do, anyway.

(c) Since the attribute grammar is one-sweep, the semantic procedure of V is easy to write. Here it is:

```
procedure V ( in β; out γ )
    variable
        β′, γ′: integer
    case rule alternative of
        "V → n , V" :  begin                          - - optimized
                           β′ := β − 1                if β > 0 then
                           call V ( β′, γ′ )              β′ := β − 1
                           if β > 0 then                  call V ( β′, γ′ )
                               γ := γ′                    γ := γ′
                           else if β = 0 then         else
                               γ := n                     γ := n
                           else                       end if
                               γ := nil
                           end if
                       end
        "V → n"      :  begin                          - - optimized
                           if β > 0 then              if β > 0 then
                               vector access error        v.a.e.
                           else if β = 0 then         else
                               γ := n                     γ := n
                           else                       end if
                               γ := nil
                           end if
                       end
    end case
end procedure
```

The procedure is written systematically, see the textbook or the lecture slides for how to do. In practice the *nil* assignment - and the whole "**else**" branch that contains it - could be omitted; but in this event the recursive call that appears in the case "$V \to n$, V", should be executed only if parameter β is not negative yet; otherwise parameter γ would just be left unassigned, which is a misbehaviour since it is output by the procedure and therefore is expected to have a vale. To optimize so properly, it suffices to take both the assignment to parameter β' and the subsequent call to procedure V, and shift them into the "**then**" branch of the "**if** $\beta > 0$" conditional, ahead of the assignment to parameter γ (see above); and to optimize the case "$V \to n$" accordingly.

Of course the semantic procedure of nonterminal E has one output parameter (the reader may wish to write this procedure as well, or see below).

```
procedure E ( out α )
    variable
        α′, α″: integer
    case rule alternative of
        "E → E + E" :  begin
                           call E ( α′ )
                           call E ( α″ )
                           α := α′ + α″
                       end
        "E → i"     :  begin
                           α := i
                       end
    end case
end procedure
```